Digital Liver World Model: Decision Memo

Architecture

I built a JEPA-style world model: an encoder maps state and context into a latent, a predictor moves it one month forward, and a constraint-aware decoder turns the prediction back into a state where the one-directional fields hold by construction. A target encoder, same weights, no gradient, runs on the true next state during training only, and the predictor learns to match what it produces, which stops it from taking the shortcut of pulling its own encoder toward it instead of learning to predict forward.

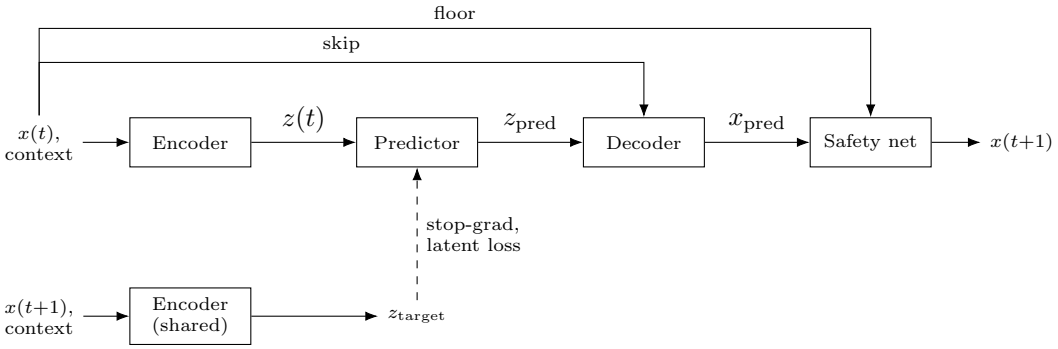

Figure 0. Model architecture. Solid arrows run at train and inference time. The bottom row only runs during training; the dashed arrow is the stop-gradient boundary, where no gradient crosses.

Where each decision landed

I did not get to run a formal ablation comparing tiers side by side. This table is the reasoning I used to choose, not a measured comparison; the ablations I would run first are in the next-steps.

Decision	Tier chosen	Why
Ratchet	Softplus increment	Clipping fixes the output after the fact and its gradient is zero exactly where the constraint is active. Softplus is positive everywhere, so the constraint holds and the gradient stays alive the whole time.
Collapse	Stop-gradient only	Chosen as the simple tier on purpose. Not fully sufficient alone, see the measured numbers on page 2.
Coupling	Explicit auxiliary loss	The generator states three couplings explicitly. Adding those as input features plus one loss term matches what is specified, without inventing structure the generator does not have.
Encoder	MLP + coupling features	A plain MLP would rediscover the multiplicative terms from scratch; graph-attention needs a real causal graph with more than three edges to earn its cost.
Multi-step training	Scheduled sampling	Pure teacher forcing never sees its own mistakes. Full backprop-through-time is more principled and is what I would try next.
Treatment / ERCP	Context-conditioned gating	The encoder sees UDCA and ERCP as flags, and the decoder branches on the ERCP flag for S. A full causal intervention model was out of scope.

Against two required alternatives

$x(t)$ **as the latent** makes the constraint story trivial but gives the predictor nowhere to represent anything not directly observable, like the hidden per-patient susceptibility scaling both the ratchet drive and the M rate. That signal has to live somewhere other than the raw 8 numbers.

A plain Neural-ODE suits continuous drift, but flares are discrete monthly events that jump the flare field and perturb A and C together, and ERCP is a discrete, non-differentiable branch in how S behaves. Forcing that into a smooth ODE means smoothing away the events or bolting on a separate handler, losing most of the benefit either way.

Collapse and Constraints

Collapse

Mechanism. Stop-gradient on the target encoder, described on the decision table, is the prevention mechanism. `CollapseMonitor` is the separate detector, run on held-out latents and independent of the loss. It reports mean and

minimum per-dimension standard deviation, and effective rank (an entropy measure over the singular values of the centered latent, which catches redundancy a per-dimension variance check alone would miss). I do not think stop-gradient alone is a complete answer: it stops the one fixed point where every input maps to the same vector, but nothing in the loss penalises a latent that only uses some of its dimensions.

Measured numbers.

	Model latent (16-dim)
Mean std	0.2816
Min std	0.1386
Collapsed dims	0 / 16
Effective rank	4.04 / 16
Verdict	Redundancy warning

Interpretation. No dimension collapsed, but the latent uses about 4.04 of its 16 dimensions.Three of the four ratchet fields, F, D, P, share the exact same monthly increment by construction, and A, C, and flare move almost in lockstep. Most of the model's 4.04 reflects the disease process itself being 3 to 4 dimensional, not the anti-collapse mechanism failing. S, uncorrelated with everything else, has not yet been checked to confirm it survives the compression on its own.

Constraints

Mechanism. The decoder produces each ratchet field as the previous value plus a positive increment,

$$x_{t+1} = x_t + \text{softplus}(h), \qquad \text{softplus}(h) = \log(1 + e^h) > 0,$$

so the constraint holds during training, not as a correction afterward. M also gets a headroom clamp so it cannot cross its upper bound,

$$x^M_{t+1} = x^M_t + \min\left(\text{softplus}(h_M),\ M_{\max} - x^M_t\right).$$

A final safety net takes a max against $x(t)$ as a hard floor, a last-line check only.

Measured violation rate (4600 held-out transitions).

Field	**Pre-safety-net**	**Post-safety-net**
F	0.0000%	0.0000%
D	0.0000%	0.0000%
P	0.0000%	0.0000%
M	0.0000%	0.0000%
S (non-ERCP months)	0.0000%	0.0000%

Pre and post match exactly: the safety net never had to correct anything, as expected given the softplus construction. An earlier version of this check reported nonzero violations purely from comparing a float32 model output against the float64 ground-truth array with too tight a tolerance; fixing the comparison to use consistent precision brought it to zero. That was a bug in how I measured the guarantee, not in the guarantee itself.

Coupling: a different question from monotonicity. Monotonicity is a shape constraint, does F ever go down. Coupling fidelity asks whether M accumulates at the right rate given F and C. The model learns one scalar, $\ell =$ `log_m_rate`, seeing the true relationship only through the auxiliary loss's gradient, never given the constant directly,

$$\Delta M_{\text{target}} = F(t) \cdot C(t) \cdot \hat{r}, \qquad \hat{r} = e^{\ell}.$$

Quantity	**Value**
Learned rate, $\hat{r} = \exp(\ell)$	0.001095
Generator's true rate	0.006000
Ratio (true / learned)	$\approx 5.5\times$
Coupling MAE (ΔM vs. true rate)	0.000398

The learned rate sits about 5.5 times below the true constant, generously, since the true rate is also scaled per patient by a hidden susceptibility factor between 0.3 and 1.0 the model never observes. The loss dropping to near zero only

means the model found one self-consistent global rate, not that it recovered the physics: monotonicity holds perfectly, accumulation fidelity does not. Next: give the encoder a short history window so it can infer per-patient susceptibility, and let the coupling rate vary per patient instead of one global scalar.

Evaluation

Example trajectory

Held-out patient: disease class 1, a treatment responder, UDCA starting month 4. History months 0 to 11: mean F 0.075, mean C 0.206, mean F×C about 0.015.

Field	F	D	S	P	A	C	M	flare
Predicted, month 20	0.086	0.067	0.141	0.704	0.1979	0.2004	0.000	0.000

Why M = 0.000. This is not the model working well. It is the coupling failure made concrete. The learned rate $\hat{r} \approx 0.001095$. For this responder patient, UDCA suppressed C to roughly 0.10–0.30 in the prediction window, giving F×C ≈ 0.010. Each predicted step increments M by $0.001095 \times 0.010 \approx 0.000011$. Over 8 prediction steps (months 12 to 20) that accumulates to ≈ 0.000088, which rounds to 0.000 at three decimal places. There are two compounding reasons: the learned rate is 5.5× too low, and this patient's F×C product is genuinely small because treatment is working. The model's explanation cites the treatment response, which is directionally correct ; a responder does accumulate M more slowly than a non-responder ; but the primary cause of the near-zero value is the rate constant failure, not the biological effect. A non-responder with the same history and same underlearned rate would also show near-zero M.

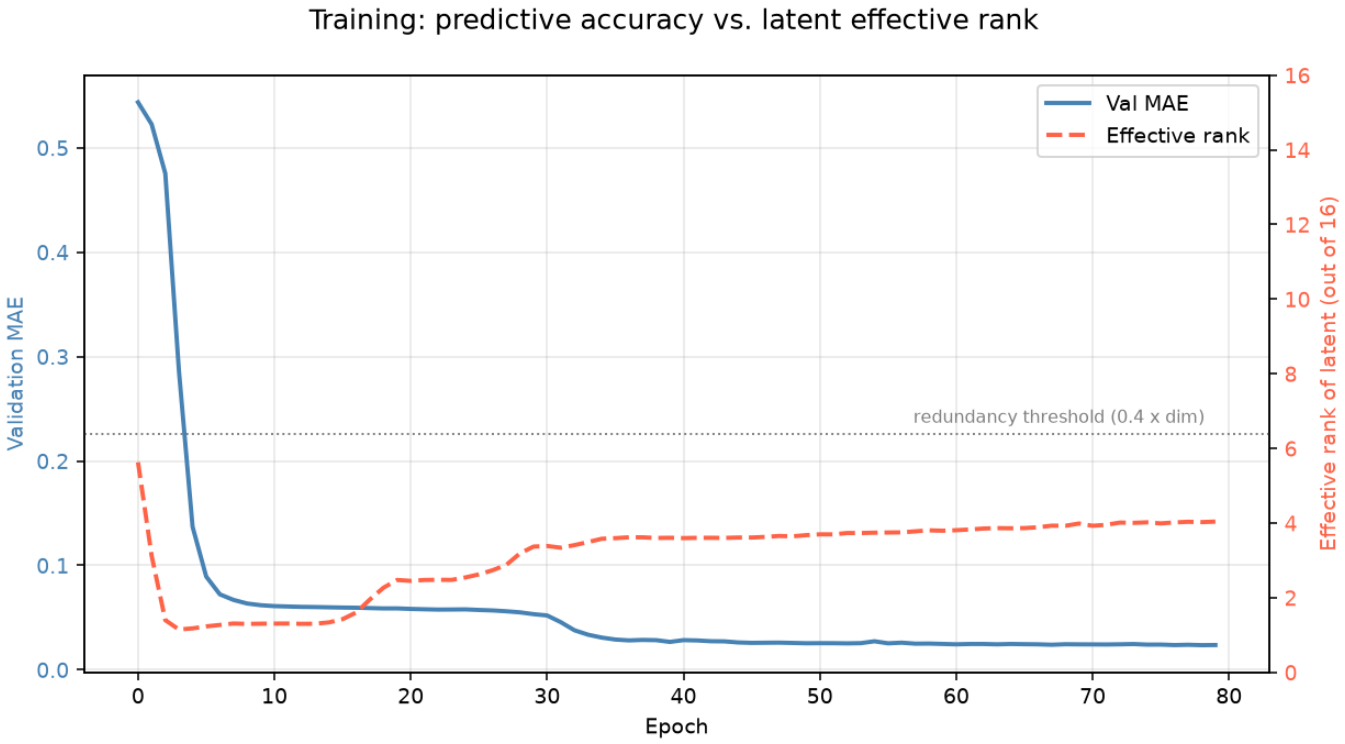

Figure 1: Training curve: validation MAE (left axis, blue) and latent effective rank (right axis, red dashed) across 80 epochs. The MAE drops sharply in the first 10 epochs as the model learns the basic ratchet structure ; F, D, P, M are monotone and have a predictable drive, so this is the "easy" part of the problem. The effective rank starts at 5.6, crashes to 1.3 by epoch 10 (this is the partial collapse visible in the training log: 6 dimensions below threshold), then slowly recovers. The crash happens because, early in training, the encoder maps all inputs toward similar latents as a shortcut, reducing the prediction error cheaply. As the reconstruction loss tightens, the encoder is forced to differentiate patients, and the rank recovers. By epoch 30 the rank stabilises around 3.4 to 4.0, below the redundancy threshold of $0.4 \times 16 = 6.4$ (dotted line), and stays there. The MAE plateau after epoch 50 at ≈ 0.024 corresponds to the irreducible error from stochastic flare events the deterministic model cannot predict, plus the systematic M underestimation from the rate constant gap. The rank never reaches the threshold because the disease process itself is low-dimensional, and stop-gradient without explicit covariance regularisation gives no incentive to spread information across unused latent dimensions.

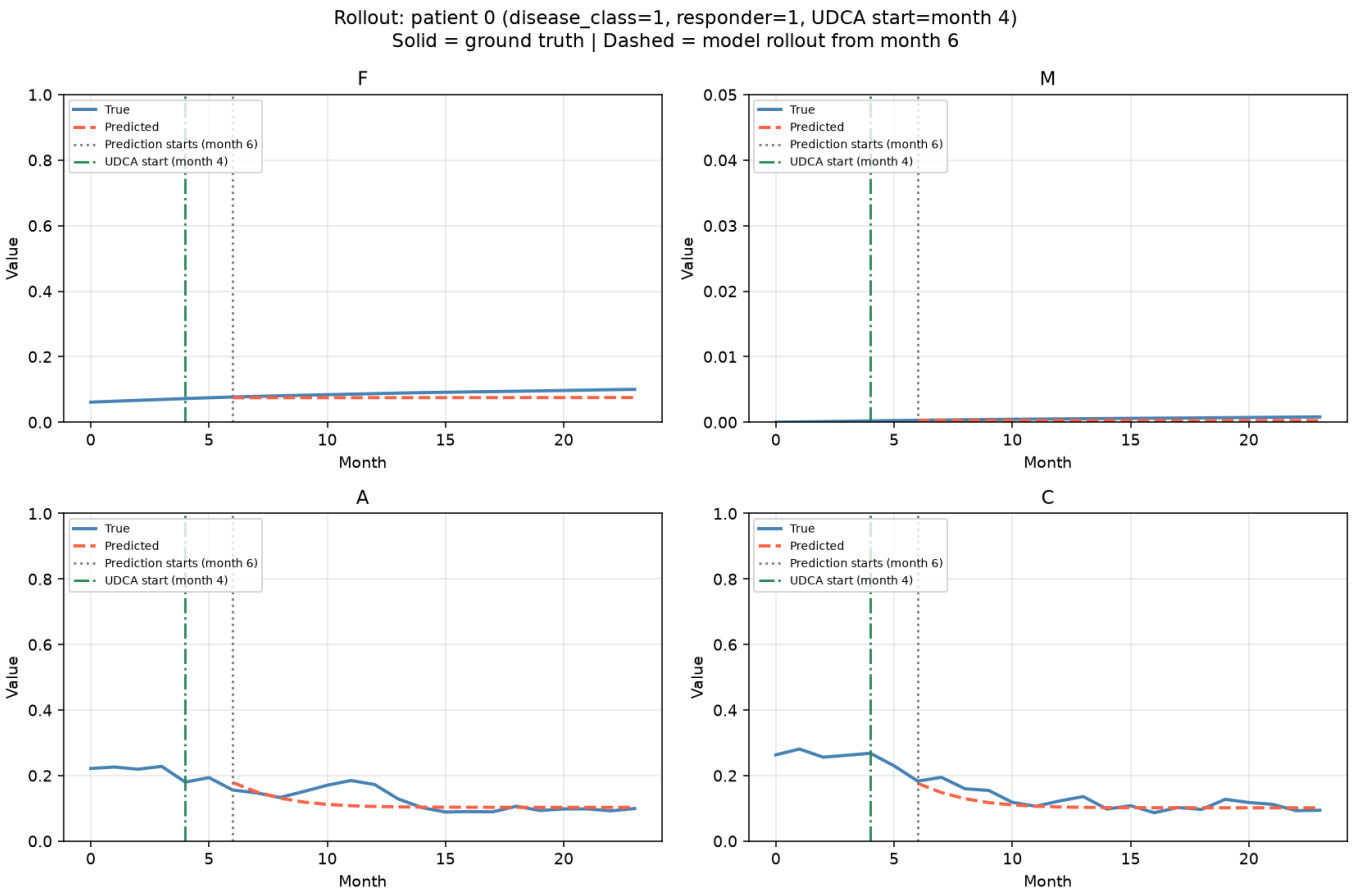

Figure 2: Rollout for patient 0: F, M, A, C, ground truth (solid blue) vs. model predictions (dashed red), months 0–24. UDCA start at month 4 (green dashed line). Prediction starts at month 6 (grey dotted line). F (top left): the predicted ratchet closely tracks the true trajectory, with small but visible drift accumulating over months 6–24. This is the model's strongest performance: the softplus construction keeps F monotone, and the ratchet drive pattern is learnable from 6 months of history. **M (top right):** the predicted M stays flat near zero for the entire rollout. The true M also stays near zero for this patient (responder, UDCA active from month 4, F×C product suppressed). However, the scale of the M panel (≤ 0.05) reveals that even the true trajectory barely accumulates ; the generator's true rate is 0.006, and this patient's suppressed F×C makes accumulation genuinely slow. The model's near-zero prediction happens to be directionally correct here, but for the wrong reason: the learned rate of 0.001095 would predict near-zero M even for a non-responder, masking the clinical distinction. **A and C (bottom row):** both fields show the mean-reverting behaviour learned correctly. The predictions converge to the setpoint of ≈ 0.10–0.11 within a few months of history cutoff and track the true mean well. However, the stochastic spikes visible in the true A and C trajectories at months 3 and 9 (flare events) are absent from the predictions ; this is expected and unavoidable: a deterministic model predicts the conditional mean, not individual realisations. The predictions are not wrong; they are forecasting the average path, not the specific sample.

Generalisation probes

Probe	Range vs. training	MAE	vs. baseline
Baseline (in-distribution)	–	0.0558	–
1: high susceptibility	0.80–1.00 vs. 0.30–1.00	0.0729	+30.7%
2: late UDCA start	months 13–18 vs. 2–8	0.0712	+27.6%
3: 48-month rollout	48 vs. 24 months trained	0.1004	+79.9%

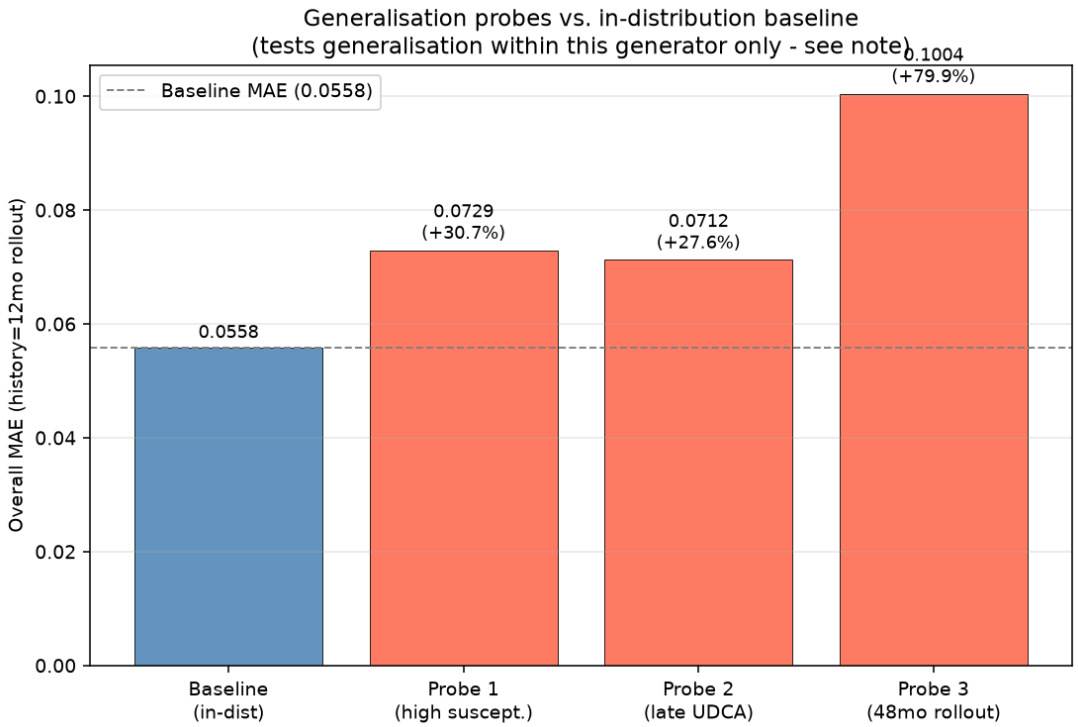

Figure 3: Generalisation probe MAE vs. in-distribution baseline (0.0558, dashed line). Percent degradation labelled on each bar. All three probes show higher MAE than the baseline, confirming the probes are testing genuinely harder conditions. **Probe 1 (+30.7%)** tests whether the model infers fast disease progression from trajectory shape. High-susceptibility patients (0.80–1.00) progress 2–3× faster through the ratchet fields. Because the encoder sees only a single timestep and cannot directly observe susceptibility, it must infer speed from context. The degradation concentrates in F, D, and P (MAE ≈ 0.038 vs. 0.027 baseline), confirming that susceptibility-driven progression is the gap. **Probe 2 (+27.6%)** tests treatment-timing generalisation. Training saw UDCA starting at months 2–8; the probe uses months 13–18. The degradation is concentrated in A and C (MAE ≈ 0.169–0.171 vs. 0.126–0.127 baseline), meaning the model has partially learned a treatment-timing prior rather than a general "suppress A and C when UDCA is active" rule. The ratchet fields degrade only modestly (+0.004), consistent with the model having learned the non-responder baseline ratchet rate reasonably well. **Probe 3 (+79.9%)** tests long-horizon stability. This is the largest degradation and it is expected for two compounding reasons: ordinary autoregressive error accumulation (each predicted step is fed back as input, drifting from the true distribution), plus the M rate-constant gap, which is a *systematic rate error* that grows linearly with horizon, not noise. At 36 months of prediction (months 12–48), the M MAE is 0.0073 vs. 0.0028 baseline ; over 2.6× worse ; directly reflecting the accumulated rate gap. Note: all three probes are within the same generator's distribution; they establish generalisation within this synthetic process, not across real disease.

What this can and cannot establish

Every result reported above comes from testing the model on data generated by the same simulator that was used for training. If the model performs well on new patients from this simulator, it means it has learned the simulator's rules. Since the simulator defines the disease in this assignment, this is the best the model can achieve. However, this does not mean the model has learned how a real liver disease works. It only shows how well the model learned this synthetic disease process. The three generalisation probes test the limits of this learning by changing susceptibility, treatment timing, and prediction horizon, but they still remain within the same simulator.